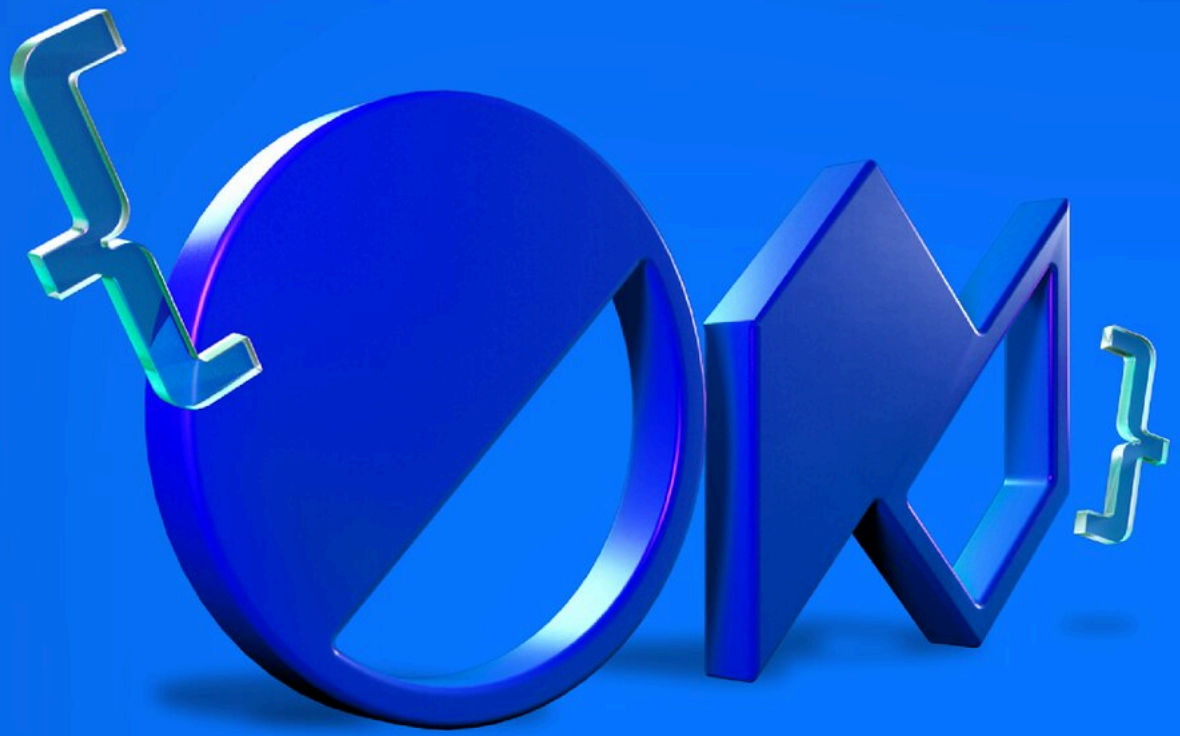
Search is test-time compute

ElasticON AI San Francisco, November 4, 2026

Han Xiao, VP of AI, Elastic



elastic

Search has mostly improved by **training harder**.

Encoders, data, indexes and rerankers all improve before the first query.



Recent LLMs found a new axis: test-time scaling



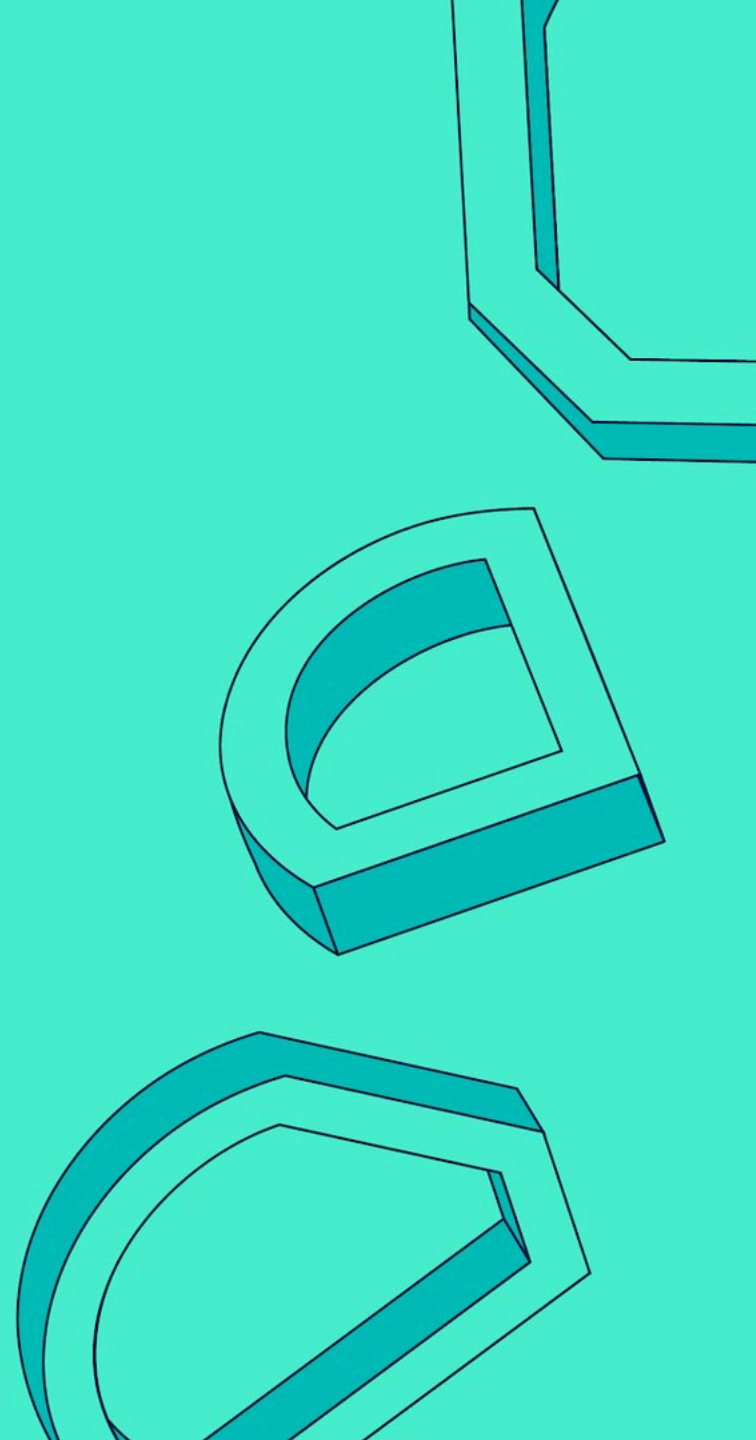
Having a bot think for just **20 seconds** in a hand of poker got the same performance boost as scaling up the model by **100,000x** and training it for 100,000 times longer.

Noam Brown, OpenAI, 2024

elastic

Fun fact: search has always lived on **that axis**.

Embedding, retrieval, reranking, expansion and fusion all run after the user presses Enter.



Inference time is a currency, not a KPI

Until now: a KPI

Inference time is a KPI

Latency is a limit written into the SLA. **Better accuracy requires a bigger model.**

becomes

In this talk: a currency

Inference time is a currency

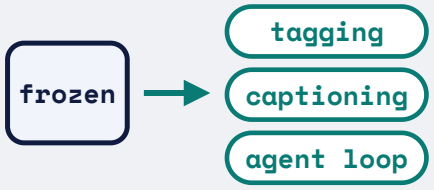
Keep the weights and raise the budget from 10 ms to 100 ms. What does it buy?



buys

Relevance

Better ranking on the same task.

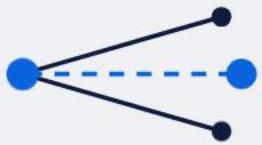


buys

A new capability

Tasks the model was never trained for.

Four test-time compute examples



01

Score functions

Higher relevance from the same frozen encoder.



02

Open-vocabulary tagger

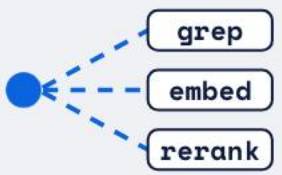
Annotate photos with any words.



03

Semantic highlighting

A reranker highlights the passages that match.



04

Reusable search pipelines

An agent builds small pipelines and reuses them in long-horizon tasks.

elastic

Three ways to spend compute budget

A Deeper pass



Read every token or patch vector of the pass you already ran.

New information: from inside the pass

B More passes



Run the encoder again on new views, such as sentences or image crops.

New information: from the input

C Calibration



Transform existing vectors with whitening, label propagation or optimal transport.

New information: from a global prior

elastic



01

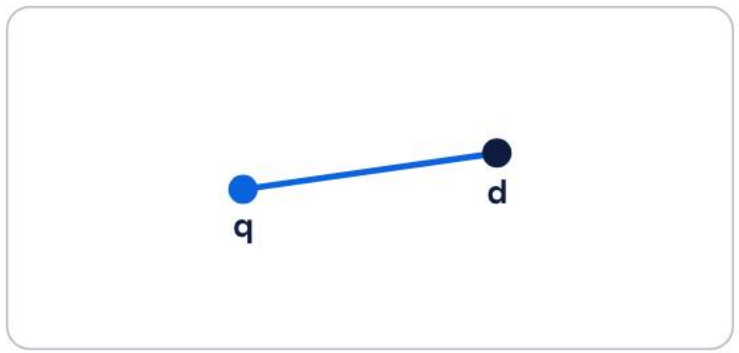
Score functions on a frozen encoder



elastic

Score function defines the relevance score

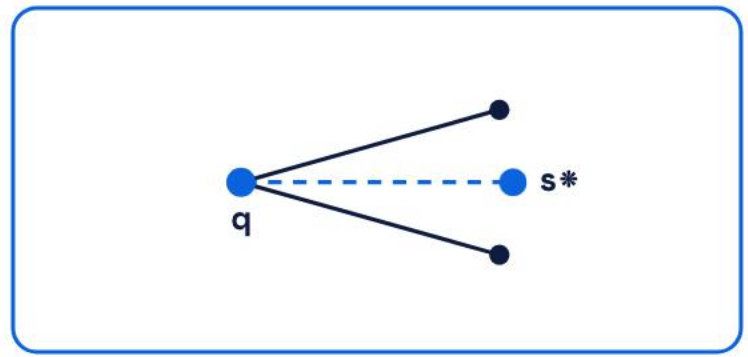
single-vector cosine



$\cos(q,\ d)$

one vector per document

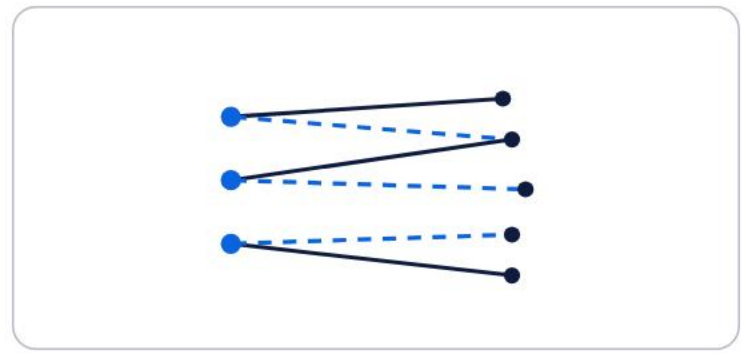
sentence MaxSim



$\max_s\ \cos(q,\ s)$

same encoder, one vector per sentence

ColBERT multi-vector

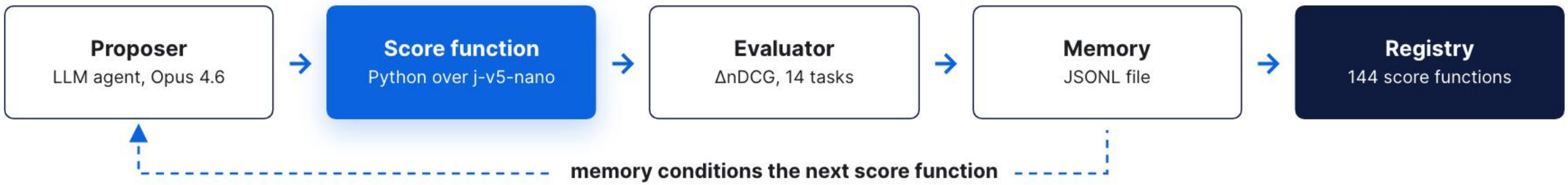


$\sum_i \max_j\ \cos(q_i,\ d_j)$

max over document tokens for each query token

elastic

Agent finds new score function in a discovery loop



writes	encoder	scores	logs	keeps
one score function per round	239M frozen	14 MMTEB tasks ΔnDCG@10	Δ, cost, parent and lesson	144 score functions

> The evaluator rejects hyperparameters, task-specific routing, external models and learned parameters.

Score function needs to be generalizable

Discovery · the loop searches here

`j-v5-nano`
239M · Jina
discovery encoder

small and fast, so each round is quick

searched over
14 MMTEB Tier-1 tasks · legal, financial, long-doc, general

→ the same score function

Held-out · the loop never sees these

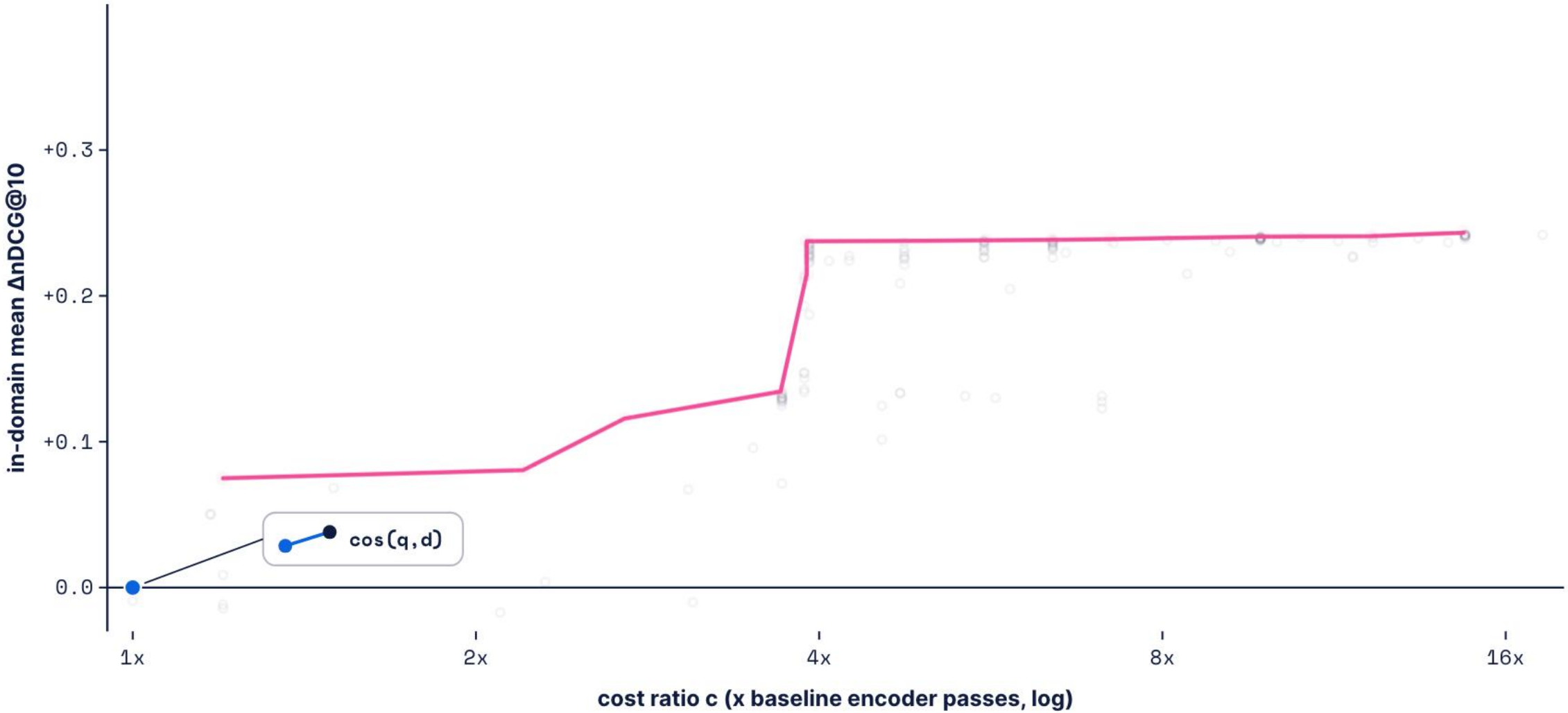
`j-v5-small`	`gemma-300m`	`qwen3-0.6b`
568M · Jina	303M · Gemma 3	600M · Qwen3
same family	**unseen family**	**unseen family**

unseen families test transfer

tested on
19 MMTEB Tier-2/3 tasks · summarization, QA, fact-check, ...

Metric: ΔnDCG@10 against the cosine baseline. **Cost c:** encoder passes per query, relative to the baseline.

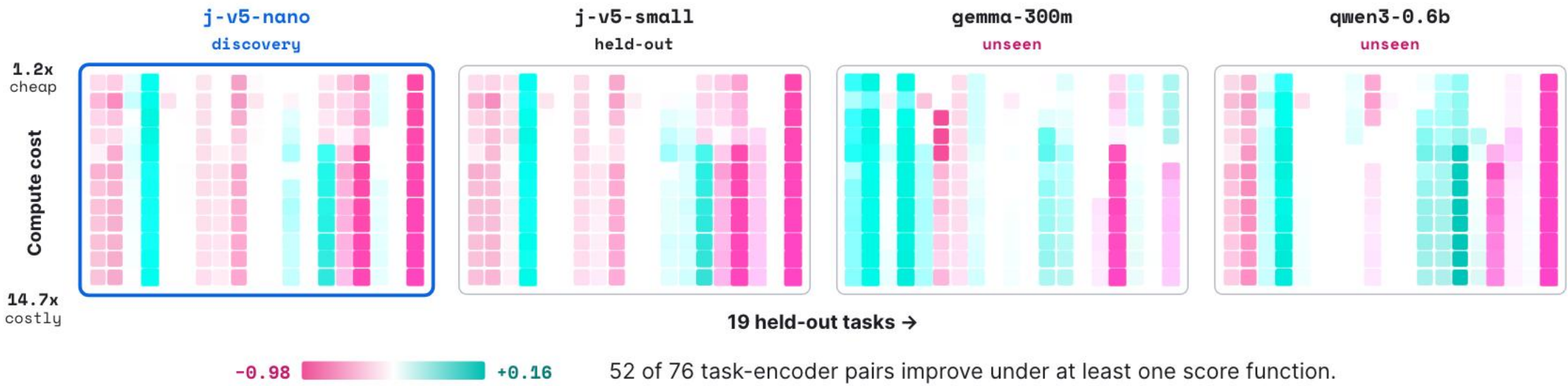
More compute, better relevance



Measured on the 14 discovery tasks.

elastic

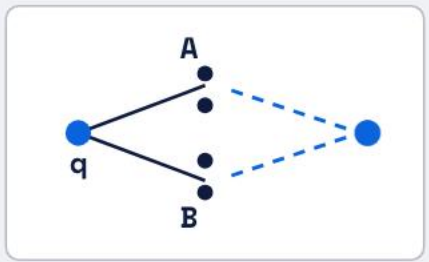
Score function generalizes over models and tasks



52 of 76 task-encoder pairs improve under at least one score function.

elastic

Final 4 winning score functions



Reciprocal Rank Fusion

appeared without a seed

fuse two rankings into one



Fisher Linear Discriminant

appeared without a seed

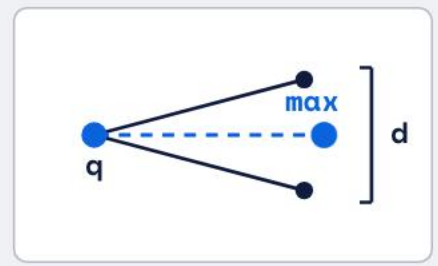
the axis that best separates relevant and non-relevant



Rocchio Pseudo-Relevance Feedback

operationalized from a seeded idea

pull the query toward the relevant centroid



Sentence-level MaxSim

operationalized from a seeded idea

score a document by its best sentence

elastic

02

A retrieval model tags images

Open-vocabulary tags from a frozen embedding model



9 photos · top-5 tags each

elastic

Use jina-embeddings-v5-omni-nano for image tagging



per image · test time

cat
coche
кот

× 128,260 tokens

vocabulary · encoded once

jina-embeddings-v5-omni-nano

Qwen3-VL vision tower
16 px patches
2×2 merge

EuroBERT-12L text tower
bidirectional · 768-d

vision tokens injected at image-token positions

frozen

P: one 768-d row per patch, attending to the *whole* image

g: pooled image vector

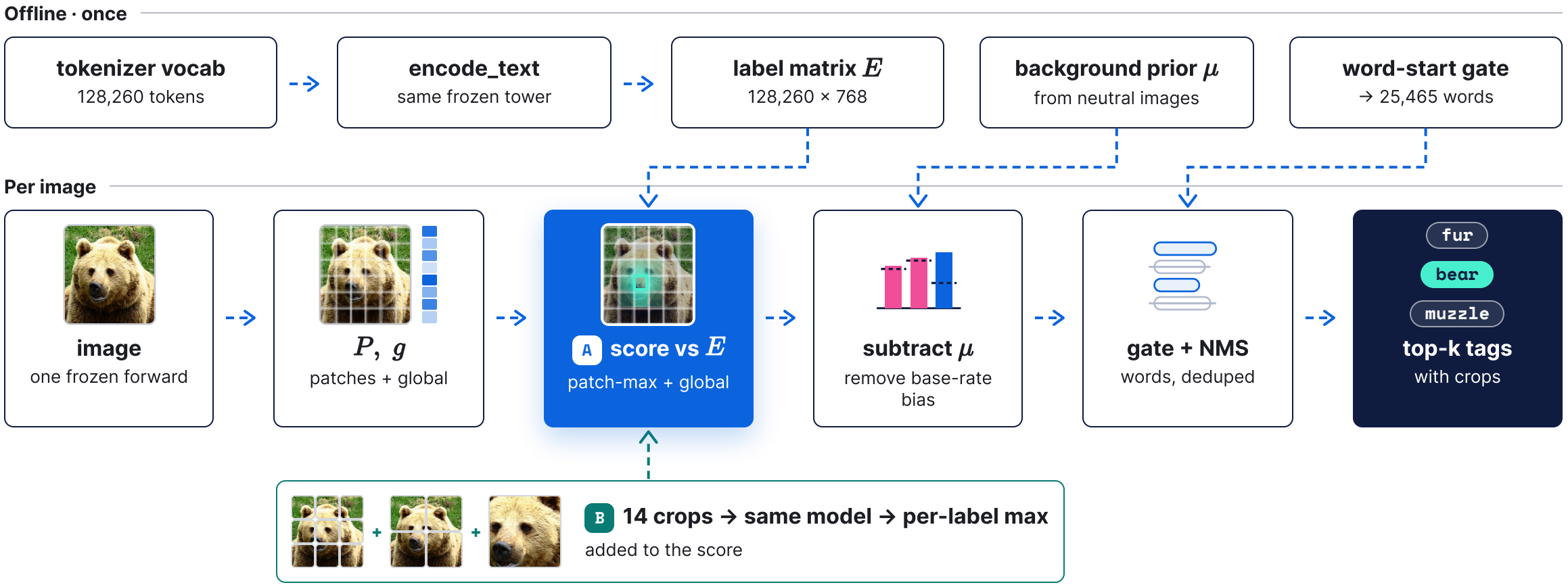
E: label matrix, 128,260 × 768

Any image patch can be scored against any word in the vocabulary.

elastic

The whole tagger is test-time compute

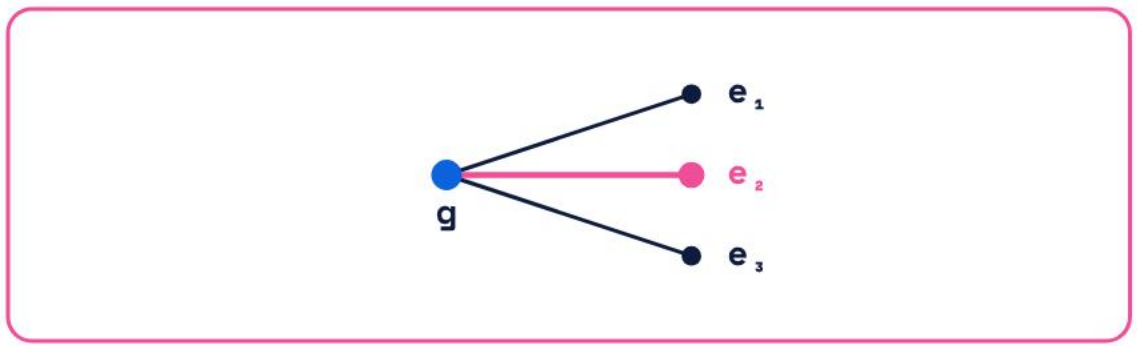
The label set is the tokenizer vocabulary: 128,260 tokens, gated to 25,465 whole words.



Every box is the frozen model or arithmetic. A reads more of one pass, B adds passes on crops.

Image as single vector or multi-vector?

global pooled

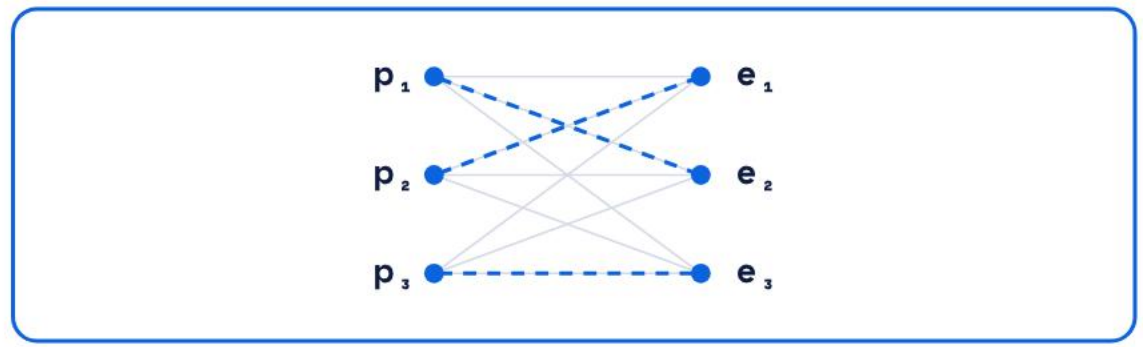


$$s_\ell = \langle g,\, e_\ell \rangle$$

one vector, dominated by the most salient object

mAP 0.264

per-label max over patches



$$s_\ell = 0.7 \max_{p \in P} \langle p, e_\ell \rangle + 0.3 \, \langle g, e_\ell \rangle$$

best patch per label, plus the global vector

mAP 0.635

elastic

Spend compute on crops



- **+0.075 mAP** for 14x the passes: 0.635 to 0.710.
- P@1 goes from **0.753** to **0.813**.
- One pass reads **wolf, roar, muzzle**. With crops it reads **fur, bear, muzzle**.

elastic

Image tagging as test-time compute

Recursion: the same scoring function runs again on each crop as a new image

$$S(\ell) = \underbrace{0.3\left(\langle g,\, e_\ell\rangle - \mu_\ell^g\right)}_{\text{C Calibration}} + \underbrace{0.7\left(\max_{p\in P}\langle p,\, e_\ell\rangle - \mu_\ell\right)}_{\text{A Patch evidence}} + \underbrace{1.3\left(\max_{c=1..14} s_\ell(\text{crop}_c) - \mu_\ell\right)}_{\text{B Multi-crop re-encode}}$$

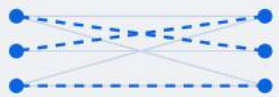
C



Calibration

The pooled vector minus its own background prior, at no extra cost.

A



Deeper pass

Arithmetic on patch rows the forward pass already produced, at no extra cost.

B

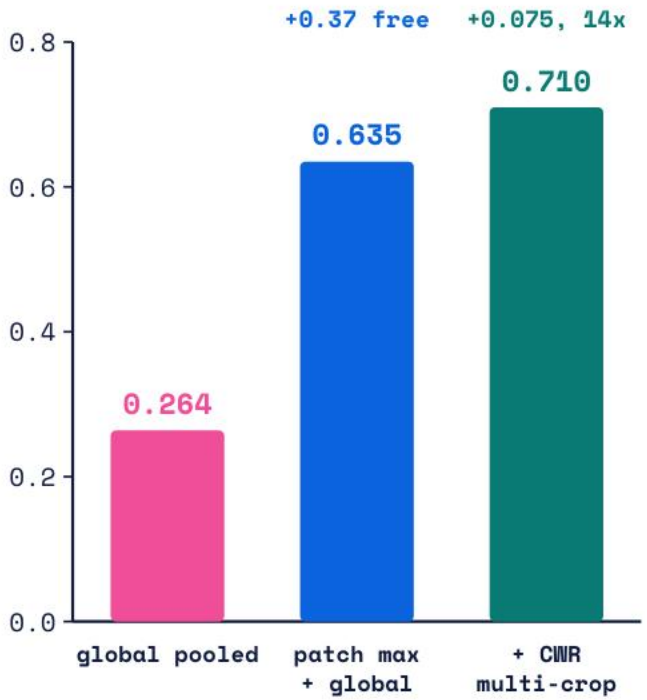


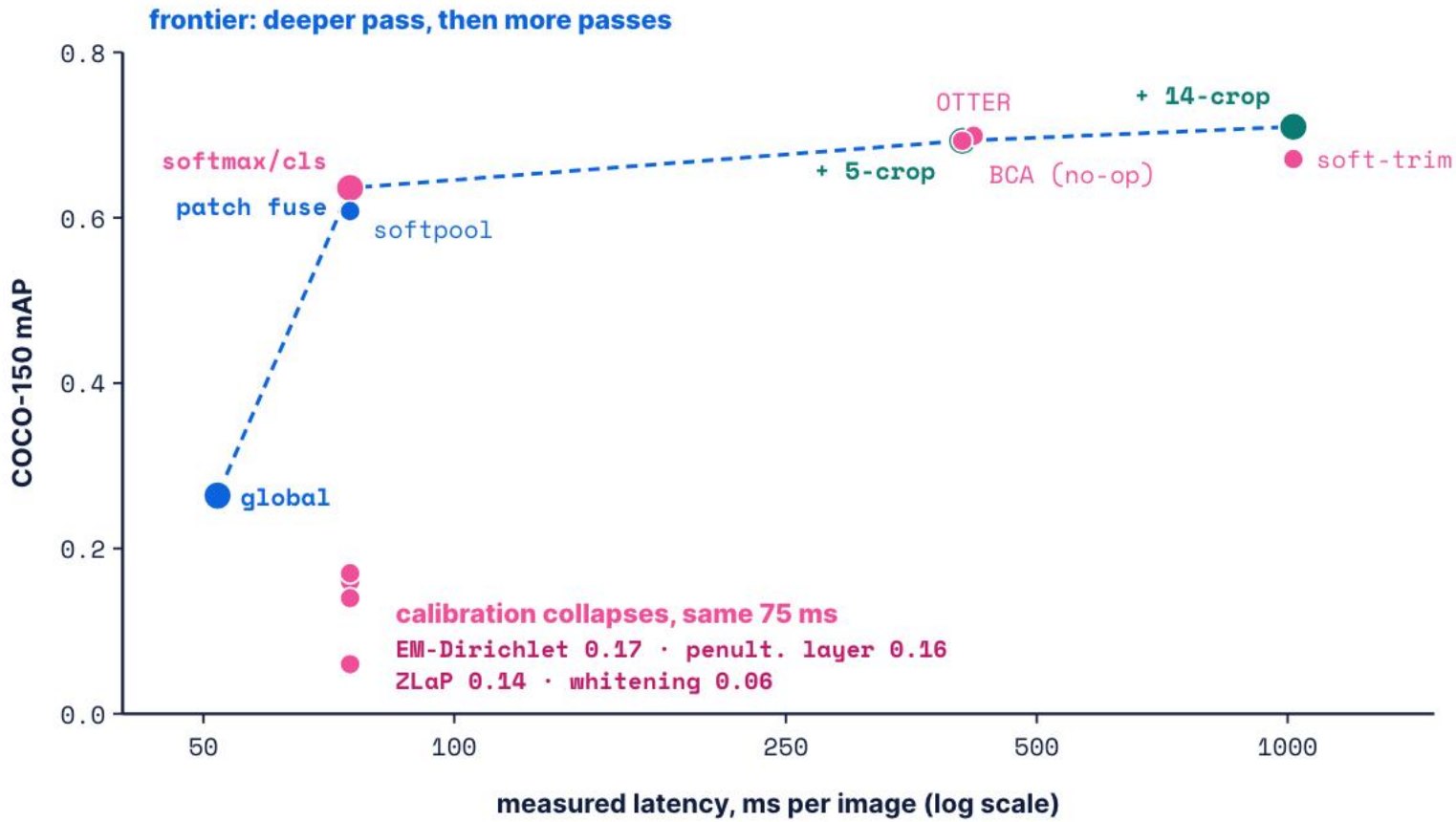
More passes

14 new forward passes over new pixels.

elastic

Accuracy climbs with more test-time compute





COCO-150 mAP: global → patch → + 14 crops

elastic

03

Reranker as semantic highlighter

elastic

Highlight the passages that answer the query

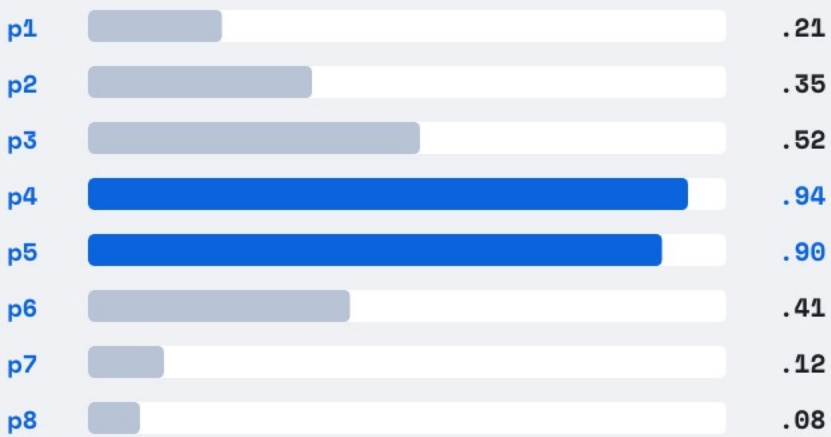
GitHub

query **Why is the sky blue?**

document

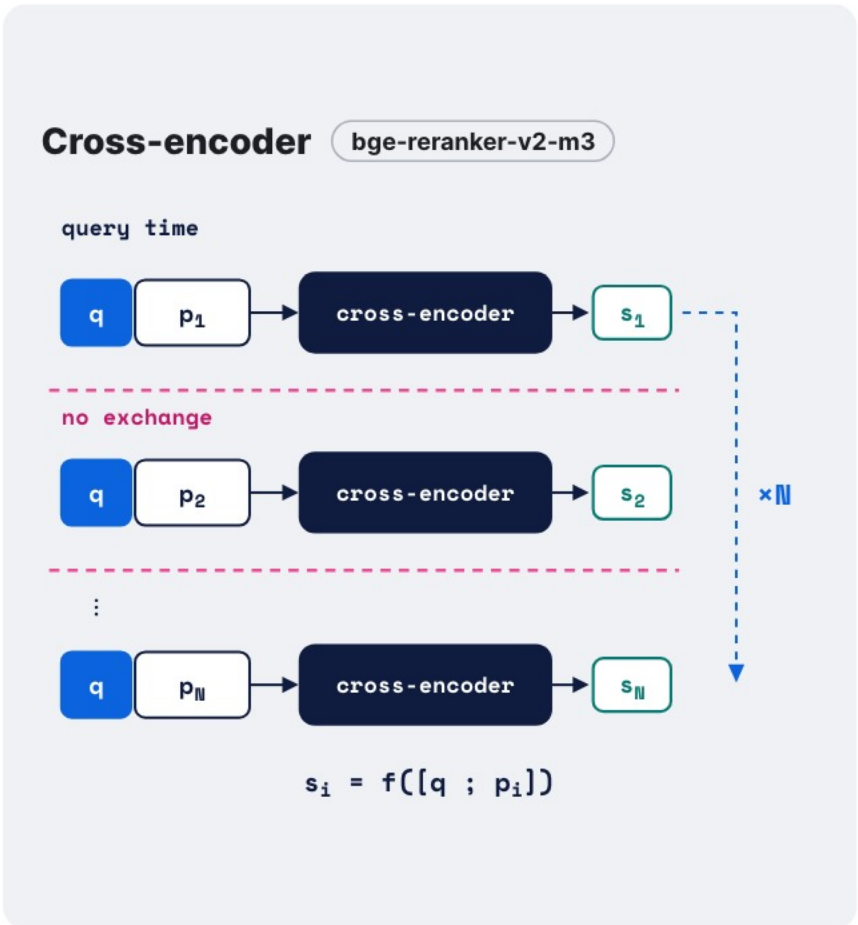
p1 The color of the sky puzzled people. p2 Sunlight carries every rainbow color. p3 Air molecules are tiny next to light. p4 They scatter blue far more than red. p5 That blue reaches us from all sides. p6 At sunset, the blue scatters away. p7 Clouds scatter all colors equally. p8 On Mars, dust turns the sky tan.

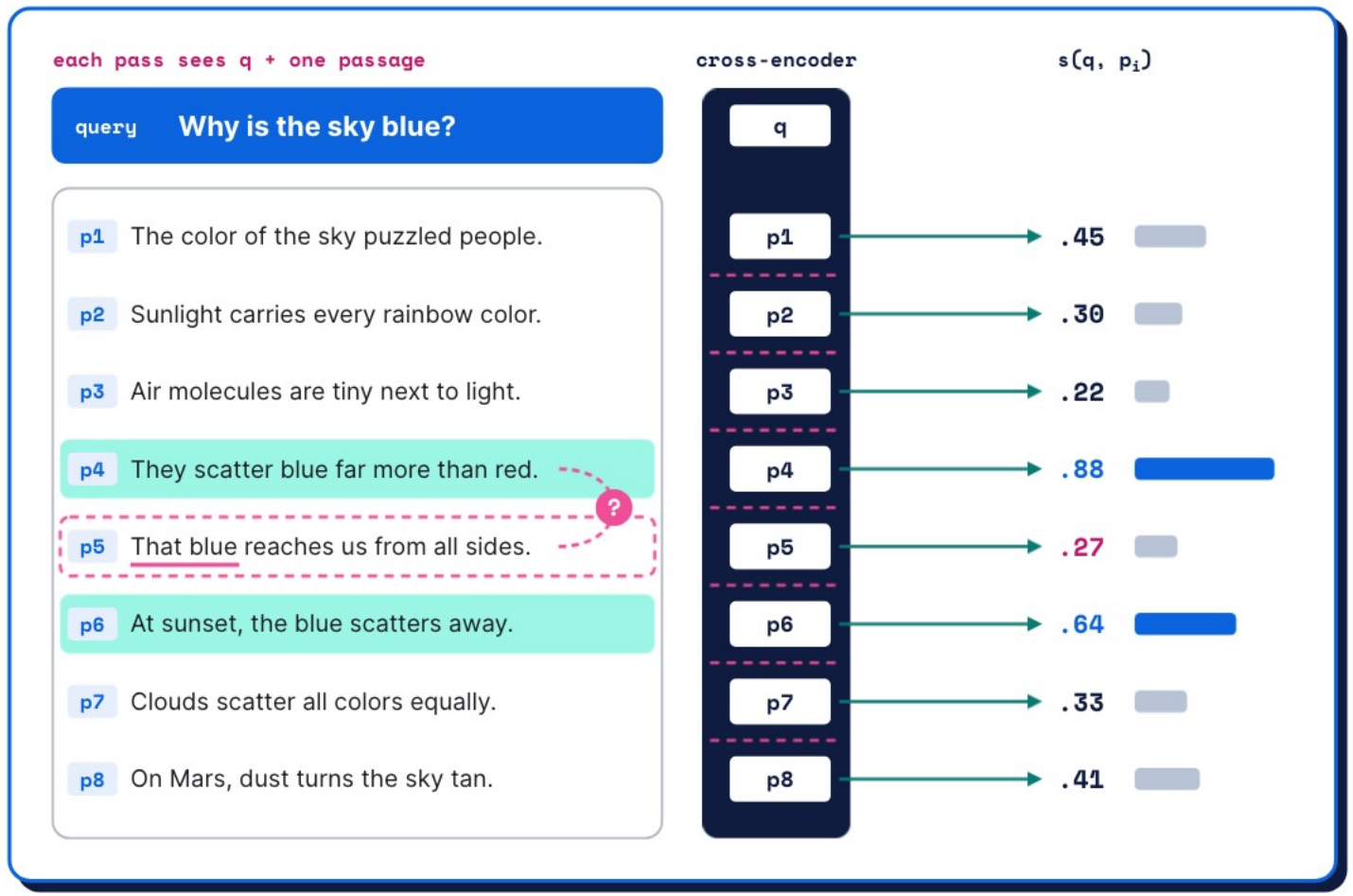
input	query q, long document D
split	D into passages $p_1 \dots p_N$
score	$s(q, p_i)$ for every passage
output	top passages, highlighted in place



elastic

Cross-encoder: score each passage alone



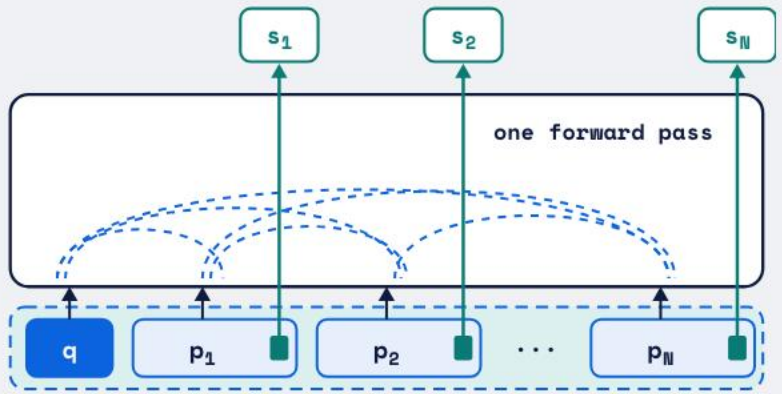


elastic

jina-reranker-v3/v3.5 new listwise reranker that scores all passages in one shot

Listwise jina-reranker-v3 · 0.6B | jina-reranker-v3.5

score = last token of each passage



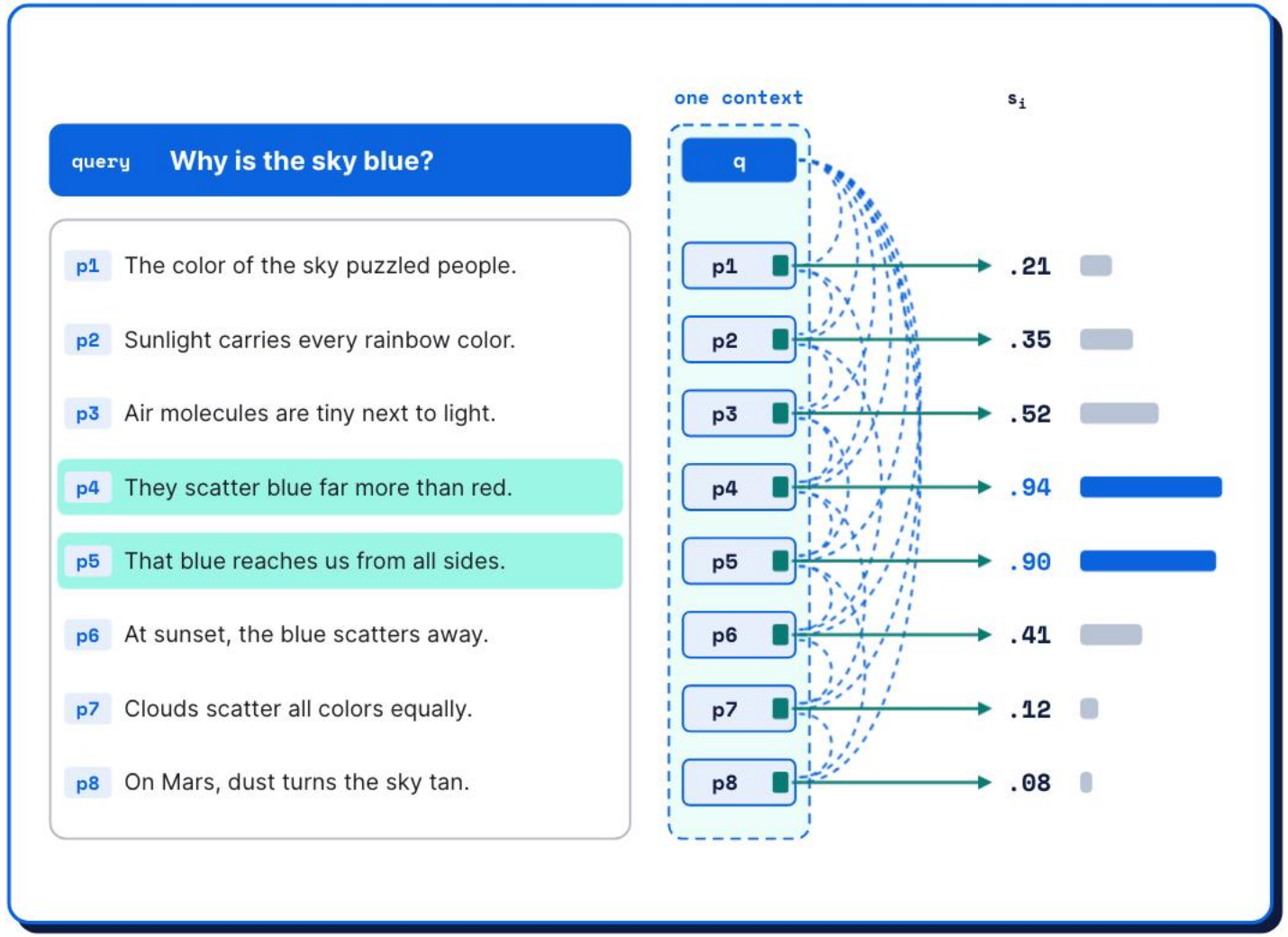
one 131K-token context · up to 64 passages

BEIR nDCG@10 · 0.6B class

jina-reranker-v3 (listwise) 61.94

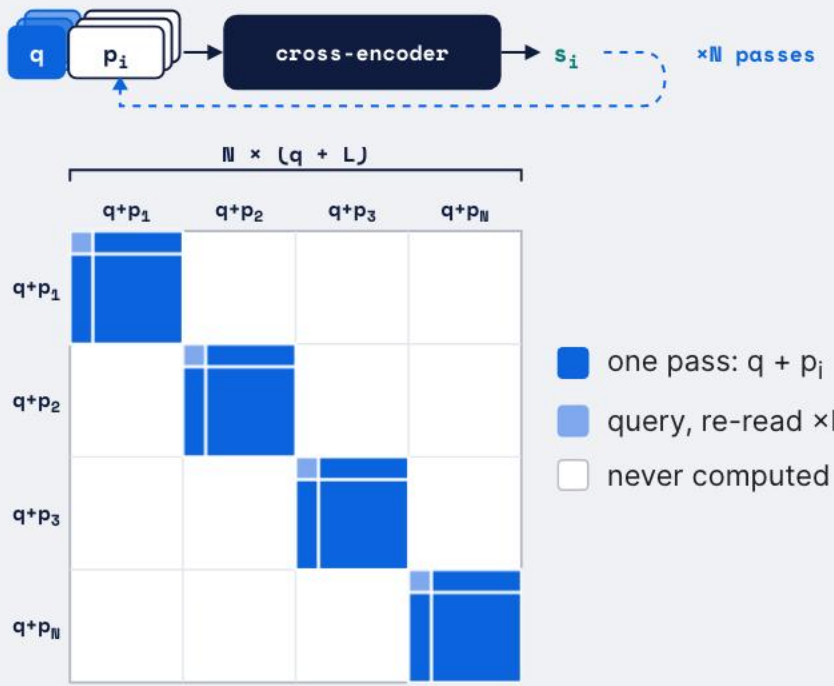
bge-reranker-v2-m3 56.51

Qwen3-Reranker-0.6B 56.28



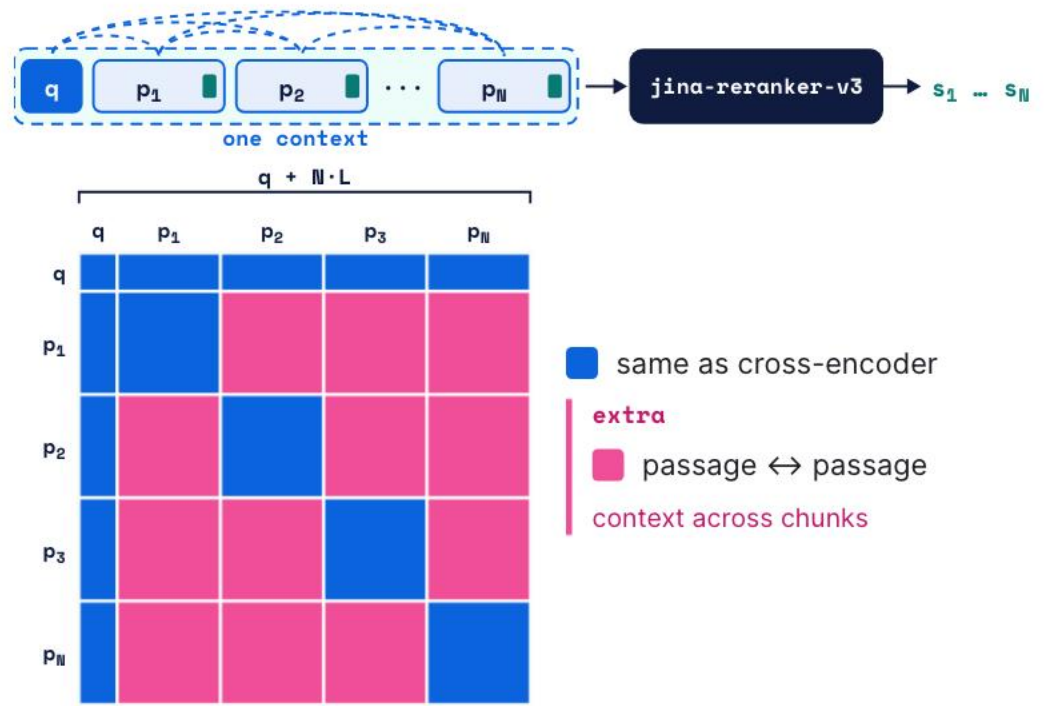
Where the extra compute goes

Cross-encoder



query time $O(N \cdot (q+L)^2) \approx O(N \cdot L^2)$

Listwise



query time $O((q+N \cdot L)^2) \approx O(N^2 \cdot L^2)$

elastic

04

Build reusable agentic pipeline at test-time

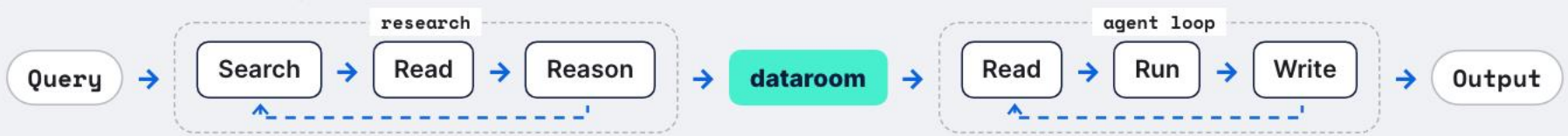
Test-time compute is the pattern for long-horizon agents

2025 · Deep research



one loop · web IO throughout

2026 · Long-horizon tasks (e.g. RSI)

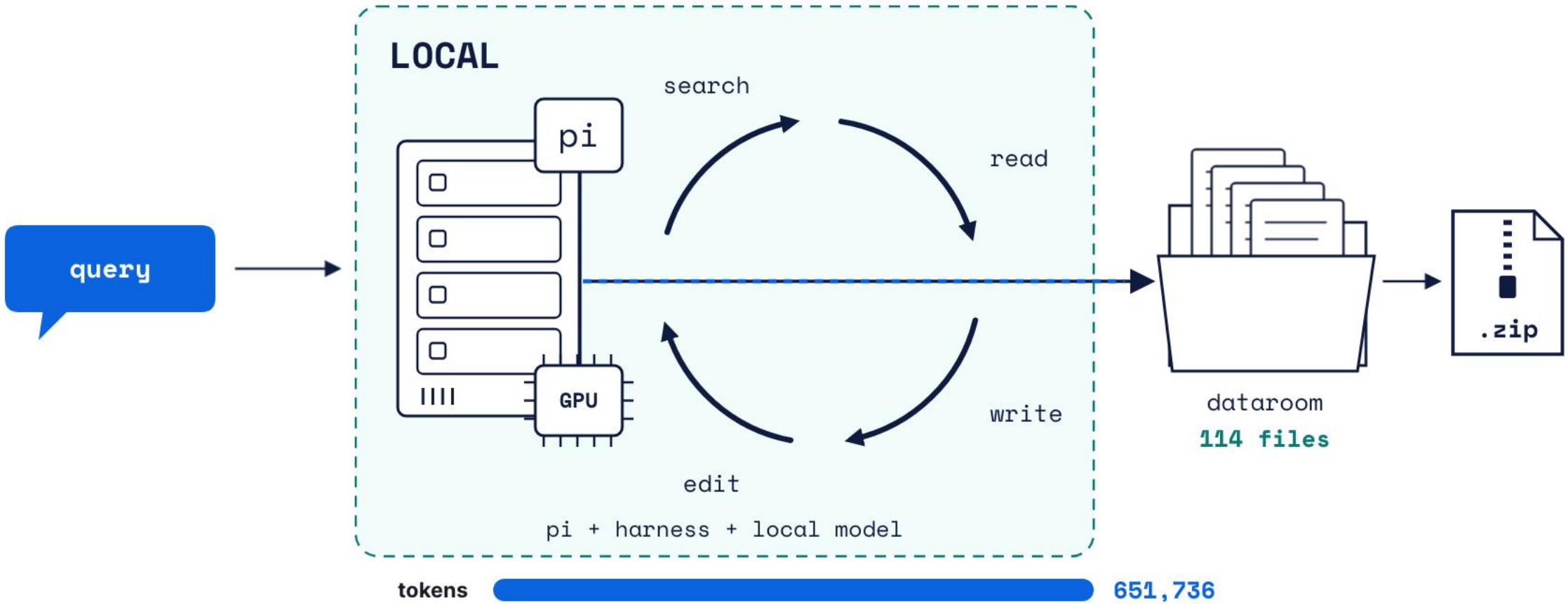


research with web IO → corpus → offline agent loop, no web IO

elastic

dataroom: a token budget builds a corpus

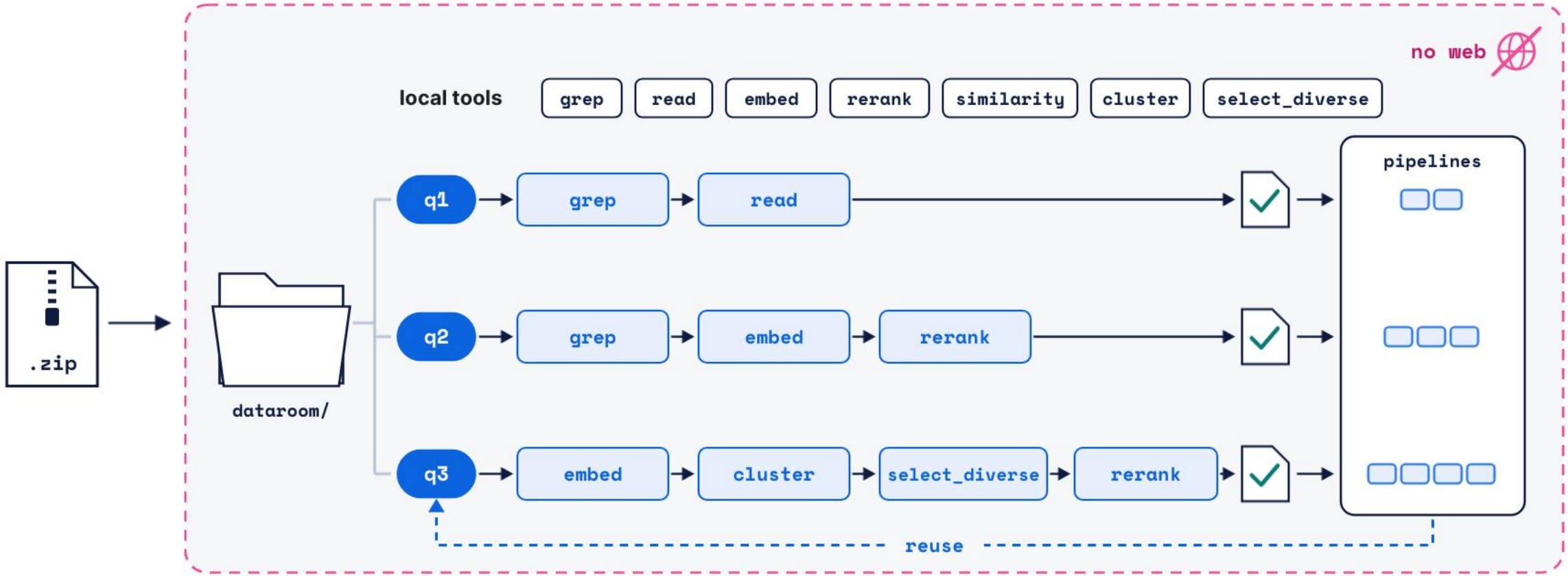
GitHub



elastic

GitHub

searchbox: an agent builds its pipeline for each query



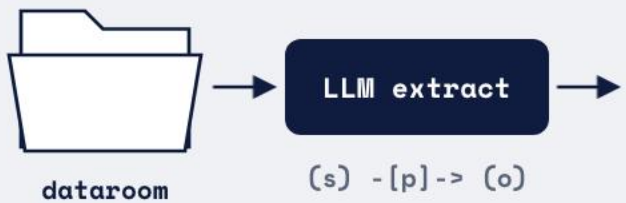
median per question, one turn: **6 tool calls · 20K tokens**

elastic

GitHub

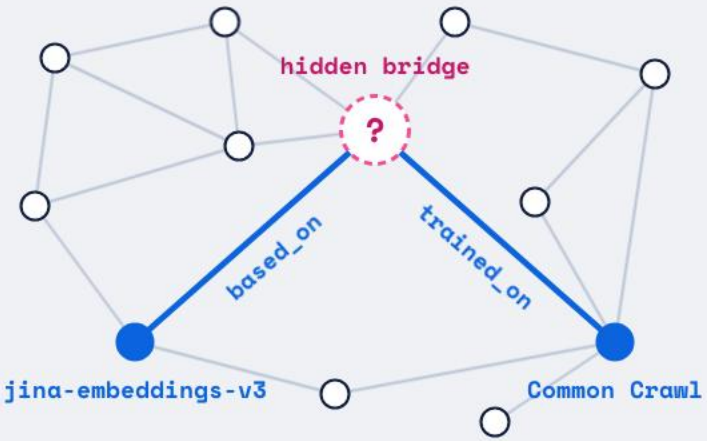
knowledge-graph: hard questions as a private verifier

If one grep finds the answer, every method scores the same. Agentic search needs questions that force several hops.



LLM extract

(s) -[p]-> (o)



Q jina-embeddings-v3 is based on a model; what was that trained on?

Common Crawl

grep + read, one turn: answered for jina-embeddings-v3 ✗

private verifier

100 questions

1 hop	76%
2–3 hops	22%

grep + read, one turn

Scores every new pipeline.

elastic

Summary of four examples



SCORE FUNCTIONS

01 **Spend compute** to build advanced score functions



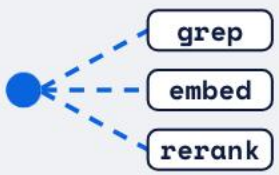
OPEN-VOCABULARY TAGGER

02 **Spend compute** to unlock a new capability: image tagging



SEMANTIC HIGHLIGHTING

03 **Spend compute** on more interactions between candidate documents



REUSABLE SEARCH PIPELINES

04 **Spend compute** to build reusable, verifiable pipelines

elastic

Search has always been test-time compute. Spend it where **new information** enters.



Thank you

Han Xiao, VP of AI, Elastic

